Documento de implementación — *Nidada* en Android, con Rust

Versión 0.1 — 6 de octubre de 2026. Pendiente de revisión.

---

0. Estatus de este documento — léelo antes que el resto

Este documento **cambia la arquitectura** del proyecto: vuelve a introducir Rust y GDExtension en el cliente, que la propuesta reducida había descartado (sección 2 de la propuesta, fila 1). La decisión está tomada por el responsable del proyecto el 6 de octubre de 2026. Este documento la desarrolla; no la reabre.

Hasta que se actualicen los documentos de abajo, este documento no se implementa. La propuesta reducida sigue siendo la única fuente de verdad, y hoy contradice a éste en estos puntos:

Documento	Punto que choca	Qué hay que cambiar
`CLAUDE.md`	Invariante 4: «Cero dependencias en `addon/`. GDScript puro. Ningún binario»	Sustituirlo por los invariantes de la sección 3 de este documento
Propuesta, sección 2	Fila « `godot-client` como GDExtension… → Addon 100 % GDScript»	Revertir la fila y justificar la vuelta (sección 1 de este documento)
Propuesta, 4.6	`addon/` en GDScript puro, cero dependencias	Nueva estructura (sección 4)
Propuesta, sección 8	Plan de 40 h, con los bloques 8 y 9 (juego) dentro	Nuevo presupuesto (sección 11): **unas 44 h**, sólo librería; el juego queda fuera del cómputo
Propuesta, sección 9	«Sin matriz de compilación»	CI con objetivos Android y Linux (sección 10)
Propuesta, sección 11	«GDExtension y cualquier código nativo» fuera de alcance	Sacarlo de la lista
`docs/preguntas-tutor.md`	Q2 sigue abierta	Añadir Q7 (sección 13)

Lo que **no** cambia: el backend sigue en TypeScript, el contrato HTTP sigue congelado tal cual, y siguen en pie los invariantes 1, 2, 3, 5, 6, 7 y 8 de `CLAUDE.md`.

Respecto a `docs/diseno-juego.md`: este documento implementa **su sección 10** (el corte vertical) en Android para el PFM, y deja escrita la arquitectura del resto del juego para después. Igual que allí, el juego se adapta a la

librería y no al revés: nada de aquí amplía el contrato HTTP.

---

1. Por qué Rust, y dónde exactamente

La propuesta quitó Rust del cliente porque su único motivo era empotrar el núcleo de transacciones en una GDExtension, y ese núcleo se fue al backend. Ese razonamiento sigue siendo correcto: **Rust no vuelve para firmar transacciones en el dispositivo**. Las claves siguen sin estar en el binario (invariante 7) y el cliente sigue sin saber que hay una cadena.

Rust vuelve por dos motivos nuevos, que la propuesta no tenía porque no tenía un juego:

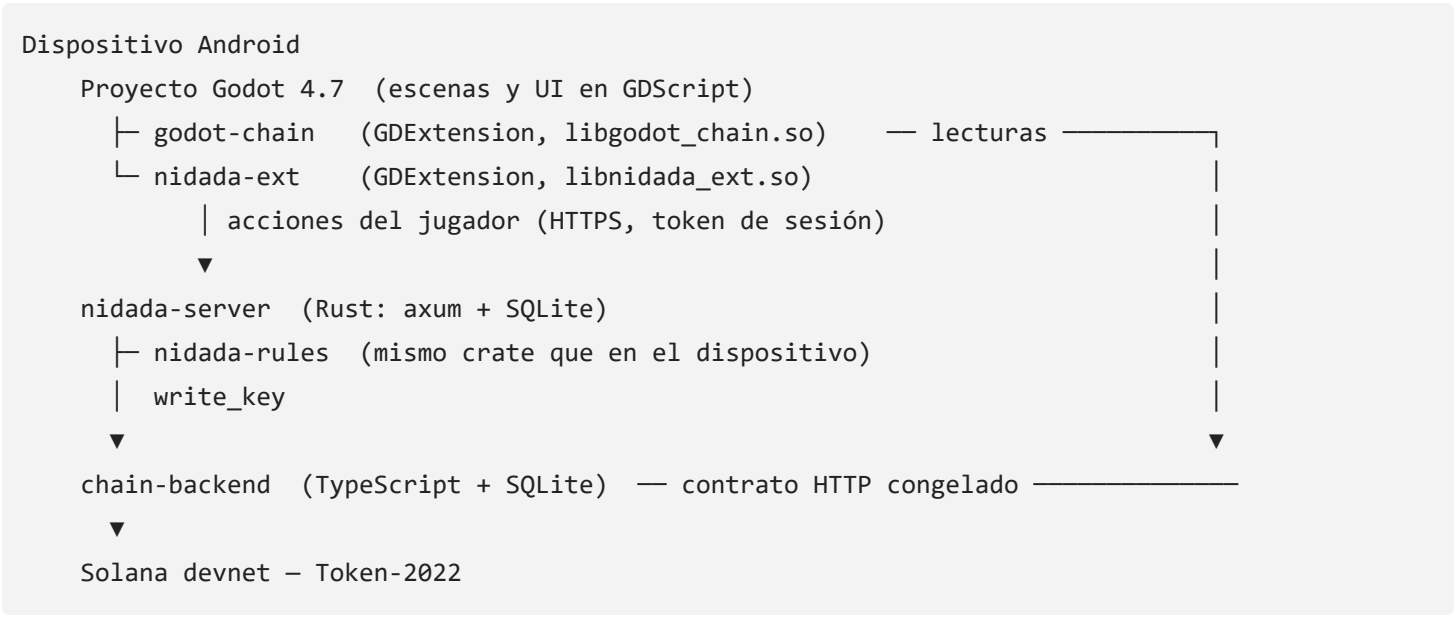
1. **Las reglas de *Nidada* se ejecutan en dos sitios.** El servidor de juego es la autoridad (diseño, sección 7), pero el cliente tiene que mostrar barras de necesidades que bajan con el tiempo, la etapa de vida y si una mascota puede criar, sin preguntar al servidor cada segundo. Con un crate de reglas en Rust compartido por servidor y cliente, la fórmula es **una sola**. Con GDScript en el cliente y otro lenguaje en el servidor, serían dos fórmulas que divergen.
2. **Requisito del proyecto.** Rust en cliente y servidor es una decisión del responsable, no una necesidad técnica de la librería. Se escribe así para que nadie la defienda con argumentos que no son los suyos.

Pieza	Lenguaje	Corre en	Motivo
`chain-backend`	TypeScript	Servidor	Sin cambios. Bloques 2–5 de la propuesta
`nidada-rules`	Rust, sin dependencias de Godot ni de red	Servidor y dispositivo	Una sola implementación de las reglas
`nidada-server`	Rust	Servidor	Servidor de juego. Sustituye al ejemplo en Node
`godot-chain`	Rust, GDExtension	Dispositivo	La librería. Sustituye al addon GDScript
`nidada-ext`	Rust, GDExtension	Dispositivo	Expone `nidada-rules` a GDScript
Escenas, UI, animación	GDScript	Dispositivo	Lo que GDScript hace bien. Rust no dibuja

Regla de reparto en el cliente: Rust calcula, GDScript muestra. Ninguna escena se escribe en Rust y ninguna regla se escribe en GDScript.

---

2. Arquitectura



Dos canales desde el dispositivo, como en la propuesta:

- **Acciones** (cuidar, regalar, proponer y aceptar cría) van a `nidada-server`, que las valida con `nidada-rules` y sus propios datos, y si tocan activos llama al backend con `write_key`.
- **Lecturas del inventario certificado** van directas de `godot-chain` al backend con `read_token`.

El dispositivo **nunca** decide nada (diseño, sección 7). `nidada-ext` en el cliente sólo predice para dibujar; cuando el servidor responde, manda el servidor.

---

3. Invariantes nuevos

Sustituyen al invariante 4 de `CLAUDE.md`. Los demás siguen igual.

1. **Cero vocabulario de cadena en `godot-chain` y en `nidada-ext`.** El `grep` de la CI se amplía a sus `src/` y a la API registrada en Godot. Rust no es excusa para dejar pasar `pubkey`.
2. **Cero crates de cadena en el dispositivo.** `cargo tree` de `godot-chain` y de `nidada-ext` no contiene ningún crate `solana-*` ni `spl-*`. La CI lo comprueba.
3. **Sin red propia ni hilos en las GDExtension.** Ni `tokio`, ni `reqwest`, ni `std::thread`. El HTTP pasa por `HTTPRequest` de Godot y todo ocurre en el hilo principal. `cargo tree` lo comprueba. `nidada-server` sí usa `tokio`: es un servidor.
4. **`nidada-rules` es puro.** Sin Godot, sin red, sin reloj del sistema, sin aleatoriedad propia. El tiempo y la semilla aleatoria entran como argumentos. Es lo que permite testearlo y usarlo igual en los dos lados.
5. **Versiones exactas** en `Cargo.toml` (`=x.y.z`), igual que en `package.json`.

---

4. Estructura del repositorio

Directorio	Contenido	Cambio
`backend/`	`chain-backend`, TypeScript	Ninguno
`rust/`	Workspace de Cargo: `godot-chain`, `nidada-rules`, `nidada-ext`, `nidada-server`	**Nuevo**
`addon/addons/godot_chain/`	`godot_chain.gdextension`, `bin/` con las `.so`, `plugin.cfg`, `plugin.gd`	Pasa de GDScript a envoltorio de la GDExtension
`examples/game-server/`	Desaparece: lo sustituye `rust/nidada-server`	**Eliminado**
`examples/godot-demo/`	Proyecto Godot de *Nidada*	Se queda, con el juego dentro
`docs/`	Contrato, diseño, este documento	—

```
rust/
├── Cargo.toml              # workspace, perfiles, versiones fijadas
├── rust-toolchain.toml     # 1.94.0 + objetivos Android
├── godot-chain/            # cdylib. Chain, ChainConfig, ChainHandle, ChainError
├── nidada-rules/           # lib. Necesidades, etapas, cría, herencia. Sin dependencias pesadas
├── nidada-ext/             # cdylib. Clases Godot finas sobre nidada-rules
└── nidada-server/          # bin. axum, rusqlite, cliente del backend
```

Por qué dos GDExtension y no una. `godot-chain` es el producto del PFM y tiene que poder usarse sin *Nidada*. Meter las reglas del juego en la misma `.so` mezclaría librería y juego, y rompería la guardia de vocabulario en cuanto el juego necesitase algo propio. El coste es compilar dos `.so` por arquitectura; con el mismo comando de Cargo es un coste de CI, no de trabajo.

---

5. Toolchain Android

Comprobado el 6 de octubre de 2026 contra la documentación de godot-rust y crates.io. Lo marcado **Verificar** no se decide: se comprueba el día que se monte.

Pieza	Valor	Nota
Godot	**4.7.2-stable** (18 de agosto de 2026)	Fijar en el README. Exportar con las plantillas de la misma versión
Crate `godot` (gdext)	**=0.5.5** (9 de agosto de 2026)	Cambia con rupturas a menudo: versión exacta, nunca rango
Rust	**1.94.0**, fijado en `rust-toolchain.toml`	Es la MSRV de `godot` 0.5.5
Característica de API	`api-4-7`	**Verificar** el nombre exacto en la documentación de gdext 0.5.5. Regla: la versión de Godot en ejecución tiene que ser ≥ la de la API compilada
Objetivos Rust	`aarch64-linux-android` (dispositivos), `x86_64-linux-android` (emulador)	No se compilan `armv7` ni `i686`: no hay dispositivo de prueba que los use
NDK	El que pida el `config.gradle` de Godot 4.7.2	**Verificar**. No mezclar NDK distintos entre Godot y Rust
JDK	17 según la guía de godot-rust	**Verificar** contra Godot 4.7
Enlazado	`cargo-ndk`, versión fijada	Alternativa documentada: `CLANG_PATH` + `CARGO_TARGET_<TRIPLE>_LINKER` a mano, como describe la guía de godot-rust

Por qué `cargo-ndk` y no las variables a mano. Las rutas del NDK dependen del sistema operativo anfitrión (`windows-x86_64` en local, `linux-x86_64` en la CI). `cargo-ndk` las resuelve a partir de `ANDROID_NDK_HOME` y deja un único comando para los dos:

```
cargo ndk -t arm64-v8a -t x86_64 -o ../addon/addons/godot_chain/bin build --release -p godot-chain
```

Si `cargo-ndk` da problemas en Windows, se vuelve al método de la guía oficial sin discutirlo.

5.1 Fichero `.gdextension`

```
[configuration]
entry_symbol = "gdext_rust_init"
compatibility_minimum = 4.7

[libraries]
linux.debug.x86_64     = "res://addons/godot_chain/bin/linux-x86_64/libgodot_chain.so"
windows.debug.x86_64   = "res://addons/godot_chain/bin/windows-x86_64/godot_chain.dll"
android.debug.arm64    = "res://addons/godot_chain/bin/arm64-v8a/libgodot_chain.so"
android.release.arm64  = "res://addons/godot_chain/bin/arm64-v8a/libgodot_chain.so"
android.debug.x86_64   = "res://addons/godot_chain/bin/x86_64/libgodot_chain.so"
```

Linux y Windows hacen falta aunque el objetivo sea Android: Linux para la suite *headless* de la CI, Windows para abrir el proyecto en el editor en la máquina de desarrollo. `nidada-ext` lleva un fichero gemelo.

5.2 Perfil de compilación

```
[profile.release]
opt-level = "s"     # tamaño antes que velocidad: el cuello de botella es la red, no la CPU
lto = true
codegen-units = 1
strip = true
panic = "unwind"    # gdext captura pánicos en la frontera; con "abort" el juego se cierra sin
mensaje
```

5.3 Exportación

- Preset Android en `examples/godot-demo/export_presets.cfg`, **sin** el sistema de compilación Gradle salvo que haga falta: una GDExtension no lo exige.
- Permiso `INTERNET` activado en el preset. Sin él, `HTTPRequest` falla en el dispositivo y funciona en el editor, que es el peor tipo de fallo.
- Orientación vertical. *Nidada* es un juego de una mano.
- Las `.so` entran en el APK por el `.gdextension`; no se copian a mano en `jniLibs`.
- **Verificar:** que las `.so` cumplen el requisito de páginas de 16 KB de Google Play para apps que apuntan a Android 15 o superior. Con NDK r28 o posterior es el comportamiento por defecto; con uno anterior hace falta pasar `-z max-page-size=16384` al enlazador.
- **Verificar:** el SDK mínimo y el SDK objetivo que fija Godot 4.7.2 por defecto. No se cambian sin motivo.

5.4 Red en desarrollo

El `localhost` del teléfono no es el del PC. Para no meter IP de la red local en la configuración:

```
adb reverse tcp:8080 tcp:8080   # nidada-server
adb reverse tcp:3000 tcp:3000   # chain-backend, lecturas
```

Con eso `ChainConfig` y la configuración del servidor de juego usan `http://127.0.0.1` igual en el editor que en el dispositivo. En el emulador vale lo mismo. **Verificar** si Android bloquea HTTP sin cifrar a través de `HTTPRequest` de Godot (Godot usa su propia pila de red, así que en principio la política de *cleartext* de Android no aplica, pero no se da por hecho).

---

6. `godot-chain` en Rust

La API pública **no cambia** respecto a la sección 6 de la propuesta: mismas tres clases, mismas señales, mismos errores, mismo modo mock. Un juego escrito contra el addon GDScript funciona igual contra éste. Lo que cambia es dónde está escrito.

Clase Godot	Rust	Base	Notas
`ChainConfig`	`#[derive(GodotClass)] #[class(base = Resource)]`	`Resource`	Campos con `#[export]`. `mode` como `enum` exportado `LIVE` / `MOCK`
`Chain`	`#[class(base = Node)]`	`Node`	Posee el pool de `HTTPRequest` como hijos. Sondea en `process()`
`ChainHandle`	`#[class(base = RefCounted)]`	`RefCounted`	Buzón pasivo. Señales `applied`, `settled`, `failed`, `finished` con `#[signal]`
`ChainError`	`#[class(base = RefCounted)]`	`RefCounted`	`category` y `retryable`. Las categorías de la propuesta, ni una más

Los cuatro problemas de la sección 6 de la propuesta siguen existiendo, y se resuelven igual:

1. **Nodo frente a `RefCounted`.** `Chain` guarda `Vec<Gd<ChainHandle>>` con los pendientes. El handle no guarda referencia a `Chain`. Sin ciclos: en Rust además lo garantiza el tipo.
2. **`await` sobre señal ya emitida.** Un handle recién creado se devuelve con estado `pending` y la primera transición ocurre como pronto en el siguiente `process()`. Nunca se emite una señal dentro de la llamada que crea el handle. Hay un test que lo comprueba.
3. **Sin hilos.** Invariante 3 de este documento. `HTTPRequest::request_completed` llega en el hilo principal; gdext se compila sin la característica de hilos experimental.
4. **Registro del autoload.** El `plugin.gd` registra como autoload una escena `chain_autoload.tscn` cuyo nodo raíz es de clase `Chain`. Un `EngineSingleton` de gdext no sirve: no es un `Node` y no recibe `process()`.

Modo mock en Rust dentro de `godot-chain`, con reloj controlable y `Chain.mock_force(handle, state, error)`. Sigue sin haber tests de conformidad frente al backend real (limitación 12.6 de la propuesta, sin cambios).

Ciclo de vida en Android. Al pasar a segundo plano Godot recibe `NOTIFICATION_APPLICATION_PAUSED` y deja de llamar a `process()`. `Chain` no hace nada especial: los handles pendientes siguen en memoria y el sondeo se reanuda al volver. Si el sistema mata la app, los handles se pierden en el cliente pero la operación sigue en el backend; al volver a abrir, el juego relee el inventario. No se persisten handles en disco: el estado que importa ya está en el servidor.

---

7. `nidada-rules`

Todo lo que el diseño define como regla y que hoy está sólo en prosa. Sin estado propio: funciones puras sobre tipos de datos.

Módulo	Contenido	Sección del diseño
`species`	Las 10 especies, su elemento, rareza y `growth_hours`. Tabla de cruce	3.2, 6.3
`needs`	Hambre, ánimo y energía en el instante `t`, a partir de la última interacción y `time_scale`	3.5
`life`	Etapa de vida y si la mascota es adulta	3.4
`breeding`	Las seis condiciones de 6.1. Devuelve el **motivo** del rechazo, no un `bool`	6.1
`inherit`	Especie y rasgos de la cría a partir de una semilla `u64`	3.3, 6.3
`breeding_fsm`	Estados y transiciones legales de una cría	6.4

La aleatoriedad entra como semilla. El servidor genera la semilla al aceptar una cría, la guarda en la fila de la cría y llama a `inherit(seed, …)`. Reintentar no vuelve a tirar (diseño 6.3) porque la semilla ya está escrita. Un PRNG determinista (`rand_chacha`, versión fijada) garantiza el mismo resultado en el servidor y en cualquier test.

Tests antes que implementación aquí, por el mismo motivo que los vectores de derivación de claves: son tablas escritas a mano en el diseño, y el test es copiarlas.

- Tabla de cruce completa (10 × 10).
- Vectores fijos de herencia: semilla conocida → color, patrón y especie esperados.
- Necesidades: valores en `t = 0`, `t = 10 h`, `t` con la mascota dormida, y suelo en 0.
- Las seis condiciones de cría, una a una, con el motivo de rechazo correcto.

---

8. `nidada-ext`

Una capa fina. Cada clase envuelve funciones de `nidada-rules` y nada más.

Clase Godot	Uso en GDScript
`PetView` (`RefCounted`)	Construida con los datos que manda el servidor. Expone `needs_at(unix_time)`, `stage_at(unix_time)`, `can_breed_with(other, now)` → motivo de rechazo o vacío
`SpeciesCatalog` (`RefCounted`)	Nombre, elemento y rareza para mostrar. No sabe nada de `item_id`: ese mapa lo da el servidor

`nidada-ext` **no** llama a la red, **no** guarda nada y **no** conoce `godot-chain`. Si GDScript quiere saber si puede pedir una cría, pregunta a `PetView` para pintar el botón, y luego el servidor decide de verdad.

---

9. `nidada-server`

Sustituye a `examples/game-server/`. Sigue siendo un ejemplo, no un producto.

Pieza	Elección
HTTP	`axum` sobre `tokio`
Persistencia	`rusqlite` con SQLite compilado dentro, un fichero
Cliente del backend	`reqwest` con `rustls`. Habla el contrato HTTP congelado, nada más
Reglas	`nidada-rules`
Credenciales	`write_key` y `read_token` desde variables de entorno. Nunca en el repositorio

En el PFM sólo implementa lo que pide el corte vertical (diseño, sección 10): crear la colección «Coleccionables», crear la Galleta de cortejo con `initial_grant` a A, y reenviar la transferencia de A a B con su `Idempotency-Key`. Sin mascotas, sin necesidades, sin cría.

Después del PFM, en este orden, porque cada paso depende del anterior:

1. Jugadores anónimos: el dispositivo pide una cuenta en la primera ejecución, el servidor crea el `external_id` y un token de sesión que el cliente guarda en `user://`. Sin login (fuera de alcance de la librería, premisa 2).
2. Individuos y necesidades: tablas `pets` e `interactions`, consumibles como contador.
3. Reconciliación (diseño 3.1) al iniciar sesión y en cada `settled`.
4. Código de amigo y visitas.
5. Cría con su máquina de estados (diseño 6.4). Regla heredada de la librería: **guardar el estado y la clave antes de llamar al backend**.

---

10. CI

Un runner Linux. La matriz vuelve, pero pequeña.

Trabajo	Qué hace
`backend`	Sin cambios: `tsc`, `eslint`, `vitest`, `npm audit`
`rust-check`	`cargo fmt --check`, `cargo clippy -D warnings`, `cargo test` de todo el workspace
`rust-guards`	`cargo tree` de `godot-chain` y `nidada-ext` sin `solana-*`, `spl-*`, `tokio`, `reqwest`. `cargo deny` para licencias y avisos
`vocab-guard`	El `grep` de siempre, ampliado a `rust/godot-chain/src` y `rust/nidada-ext/src`
`android-build`	`cargo ndk` para `arm64-v8a` y `x86_64`, en `release`. Sólo comprueba que enlaza
`godot-headless`	Compila las GDExtension para `linux-x86_64` y ejecuta la suite de `godot-chain` en modo mock con Godot 4.7.2 *headless*

No entra en la CI: generar el APK firmado, ni ejecutarlo en un emulador. Las dos cosas son lentas y frágiles en un runner gratuito. Se hacen a mano antes de la demo y se documentan con capturas.

Licencia de gdext: MPL-2.0, copyleft por fichero. Enlazarlo no obliga a nada; parchearlo obliga a publicar esos ficheros. No se parchea.

---

11. Plan y presupuesto

El tiempo de desarrollo del juego no cuenta en las horas del PFM. El presupuesto de horas cubre sólo la librería: backend, `godot-chain`, su CI y su documentación. *Nidada* (servidor de juego, escenas, arte, reglas) se hace aparte, aunque el hito lo necesite para demostrarse.

Los bloques 1–5 de la propuesta (backend) no cambian: 21 h. Lo que cambia es del bloque 6 en adelante.

Horas del PFM — la librería:

Fase	Contenido	Sustituye a	h
A0	**Prueba de vida**: GDExtension mínima de Rust con una clase que imprime un texto, cargada en Godot 4.7.2 en un teléfono real	—	4
A1	`godot-chain` en Rust: clases, pool de `HTTPRequest`, sondeo con backoff, errores	Bloque 6 (7 h)	10
A2	Modo mock y `mock_force` en Rust, suite *headless*	Bloque 7 (3 h)	4
A5	CI de la sección 10	—	2
—	README, documentación y vídeo	Bloque 10 (3 h)	3

Total del PFM: 21 h + 23 h = unas 44 h, frente a las 40 h de la propuesta.

Fuera del cómputo — el juego:

Fase	Contenido	Sustituye a	h estimadas
A3	`nidada-server` mínimo: corte vertical	Bloque 8 (2 h)	5
A4	Escena del corte vertical de *Nidada*, exportada y ejecutada en Android	Bloque 9 (3 h)	5

Se estiman para planificar el calendario, no para el presupuesto. Siguen siendo necesarias: sin A3 y A4 no hay hito que enseñar.

Comparación justa con la propuesta. Sus 40 h incluían los bloques 8 y 9 (5 h), que eran trabajo de juego. Quitándolos, la librería costaba 35 h en GDScript y cuesta 44 h en Rust: **9 h más** por Rust en el cliente. Contra el límite de 40 h se pasa en 4 h; hay que confirmar con el tutor que se aceptan o aplicar la línea de corte 8.1 desde el principio (Q7, sección 13).

`nidada-rules` y `nidada-ext` **no** están en este plan: el corte vertical no tiene mascotas. Se escriben después del PFM, igual que el resto del diseño, y tampoco cuentan en las horas.

11.1 Plan B, con disparador

Riesgo	Disparador	Plan B
La GDExtension no carga en Android	A0 supera **4 h** sin la clase funcionando en un teléfono	Volver al addon GDScript de la propuesta, sin cambios. Rust se queda sólo en `nidada-server`. Se decide al acabar A0, nunca después
El NDK de Godot y el de Rust no casan	Errores de enlace que no se resuelven en 1 h	Fijar el NDK que pide Godot y compilar Rust con ése exclusivamente
`cargo-ndk` falla en Windows	Primer intento fallido	Variables de entorno a mano, como la guía de godot-rust
Rupturas de API de gdext	Un ejemplo de la documentación no compila con 0.5.5	Leer el *changelog* de 0.5.5; no subir de versión a mitad del proyecto

A0 va **antes** que nada más de esta lista porque es la que puede tumbar la decisión entera.

---

12. Fuera de alcance

Además de la sección 11 de la propuesta y la sección 11 del diseño:

- iOS. La arquitectura no lo impide, pero no se compila, no se prueba y no se promete.
- Notificaciones *push* («tu cría ha nacido»). Exigirían Firebase y una cuenta de Google Play configurada. El jugador lo ve al abrir el juego.

- Juego sin conexión. Sin red, las lecturas acaban en `NETWORK` y la UI lo dice. Se puede mostrar el último inventario asentado guardado en `user://`, marcado como antiguo; nunca se actúa sobre él.
- `armv7` y `x86` de 32 bits.
- Publicación en Google Play. Se distribuye el APK de depuración.

---

13. Preguntas abiertas

No las responde quien implementa.

#	Pregunta	Por qué importa
Q7	¿Se aceptan unas 44 h de librería en lugar de 40 h por usar Rust en el cliente, o se aplica la línea de corte 8.1 desde el principio?	Sin respuesta, el plan de la sección 11 no tiene presupuesto
—	¿Q2 de `preguntas-tutor.md` (backend en Rust) cambia con esta decisión?	Si el tutor pide también el backend en Rust, son otras 8–12 h y el contrato HTTP seguiría igual, pero `chain-backend` se reescribe
—	¿La demo final se hace en un teléfono, en el emulador, o en escritorio con el APK como prueba aparte?	Decide si A4 incluye grabar la pantalla del teléfono
—	Política vigente de Google Play sobre contenido basado en blockchain (punto 13.8 de la propuesta)	Aunque no se publique, el README tiene que avisar con el texto actual

---

14. Verificar antes de escribir código

Comprobar contra la fuente y anotar el resultado **con la fecha**.

Ya verificado el 6 de octubre de 2026:

1. La versión más reciente del crate `godot` es la 0.5.5, publicada el 9 de agosto de 2026, con MSRV Rust 1.94.
2. Godot 4.7.2-stable es la versión estable más reciente (18 de agosto de 2026).
3. godot-rust documenta Android con los objetivos `aarch64-linux-android`, `x86_64-linux-android`, `armv7-linux-androideabi` e `i686-linux-android`, compilados por separado, con `CLANG_PATH` y el enlazador por objetivo, y entradas `android.debug.<arch>` en el `.gdextension`.
4. Regla de compatibilidad de GDExtension: carga en cualquier Godot cuya versión sea ≥ la de la API con la que se compiló.
5. En la máquina de desarrollo hay `rustc` 1.94.0 con sólo el objetivo `x86_64-pc-windows-gnu`. Faltan los objetivos Android y el objetivo MSVC si el editor de Godot lo necesita.

Pendiente:

6. Nombre exacto de la característica `api-4-7` en gdext 0.5.5.
7. NDK y JDK que exige Godot 4.7.2.
8. Requisito de páginas de 16 KB de Google Play y si el NDK elegido lo cumple por defecto.
9. SDK mínimo y objetivo por defecto de Godot 4.7.2.
10. Si `HTTPRequest` de Godot en Android permite HTTP sin cifrar hacia `127.0.0.1` con `adb reverse`.
11. Si la toolchain `windows-gnu` sirve para la `.dll` del editor o hace falta `windows-msvc`.